

Hybrid Infrastructure Automation

Infrastructure as Code for Multi-Cloud Provisioning

Global Retail Platform — Hybrid Cloud Modernization | IBM Cloud + AWS | March 2026

1. Introduction

Infrastructure as Code (IaC) enables the Global Retail Platform to provision and manage infrastructure across hybrid cloud environments through declarative, version-controlled templates rather than manual, error-prone processes. By codifying every network segment, compute cluster, storage volume, and security policy in Terraform configuration files stored in Git, the platform can guarantee that the development, staging, and production environments are identical in structure, that every change is reviewed and approved before it reaches infrastructure, and that any environment can be re-created from scratch in minutes. For the Global Retail Platform, this matters because the architecture spans two cloud providers — IBM Cloud (primary production and EU-GDPR compliance region) and Amazon Web Services (secondary disaster recovery region) — as well as on-premises data centers connected via IBM Cloud Direct Link 2.0. Maintaining consistency across these environments manually would introduce configuration drift, compliance gaps, and operational risk. IaC eliminates this risk by making infrastructure a software artifact: testable, auditable, and repeatable across every provider and every region.

This document describes the IaC design for the Global Retail Platform, covering infrastructure component definitions, reusable module architecture, state and environment management, drift detection controls, and provisioning validation procedures. All configuration examples reference Terraform HCL syntax, reflecting the tool selected for its mature multi-cloud provider ecosystem and first-class support for IBM Cloud and AWS resources.

2. Infrastructure Components

The Global Retail Platform infrastructure is decomposed into four foundational component categories: virtual networking, compute resources, storage systems, and security access controls. Each category maps to a dedicated Terraform configuration block and a reusable module. The table below identifies every core component provisioned across the hybrid environment, its purpose, and the provider responsible for hosting it.

Component	Purpose	IBM Cloud	AWS (DR)
Virtual Network (VPC)	Provides isolated network connectivity between all workloads within each cloud region; enforces segment boundaries across DMZ, application, data, and management tiers	IBM Cloud VPC — US-South & EU-Frankfurt	AWS VPC — us-east-1 (DR region)
Subnets & Route Tables	Segments traffic into distinct functional zones: public DMZ (10.0.1.0/24), API gateway zone (10.0.2.0/24), application services zone (10.0.10.0/22),	IBM Cloud Subnet resources per zone	AWS Subnet resources per AZ

Component	Purpose	IBM Cloud	AWS (DR)
	payment isolation zone (10.0.20.0/24), data persistence zone (10.0.30.0/24)		
Kubernetes Cluster (IKS / EKS)	Hosts all customer-facing microservices (Product, Order, Checkout, Customer Profile, API Gateway) with multi-zone worker pools; supports Horizontal Pod Autoscaling from 3 to 20 replicas under peak load	IBM Kubernetes Service (IKS) — multi-zone worker pools across 3 AZs	Amazon EKS — DR standby cluster, activated on failover
Compute Instances (Worker Nodes)	Provides underlying compute capacity for IKS / EKS worker pools; instance types selected to meet CPU and memory requirements for baseline and scaled workloads at 50,000 concurrent users	IBM Cloud Virtual Server Instances (bx2-8x32 profile)	AWS EC2 instances (m5.2xlarge) in Auto Scaling Groups
Managed Relational Database (PostgreSQL)	Stores transactional records for orders, customer profiles, and checkout state with ACID compliance; requires consistent low-latency IOPS to meet the sub-2-second checkout SLA	IBM Cloud Databases for PostgreSQL — synchronous 3-zone replication	Amazon RDS for PostgreSQL — Multi-AZ standby in DR region
In-Memory Cache (Redis)	Absorbs high-frequency product catalogue reads and session token lookups; prevents read amplification reaching the PostgreSQL tier during Black Friday peak load (target: >80% cache hit rate)	IBM Databases for Redis — HA cluster with leader election	Amazon ElastiCache for Redis — DR standby, warm-up from RDS on activation
Object Storage	Stores build artifacts, SBOM files, security scan reports, application logs, and DR snapshots with immutable retention policies; satisfies audit evidence requirements for GDPR and PCI-DSS	IBM Cloud Object Storage — Standard + Archive tiers, WORM buckets	Amazon S3 — cross-region replication target for DR artifacts
Block Storage Volumes	Provides persistent, high-IOPS volumes for PostgreSQL data directories and inventory state; 10 IOPS/GB tier for checkout DB; 5 IOPS/GB tier for inventory and order state	IBM Cloud Block Storage — BYOK encryption with IBM Key Protect	AWS EBS gp3 volumes — encrypted with AWS KMS
Security Groups & Network Policies	Controls inbound and outbound traffic at the instance and pod level; enforces default-deny between segments; restricts payment namespace egress to approved processor endpoints only	IBM Cloud Security Groups + Kubernetes NetworkPolicy (Calico CNI)	AWS Security Groups + EKS NetworkPolicy

Component	Purpose	IBM Cloud	AWS (DR)
IAM Roles & Access Policies	Defines least-privilege access for human operators, CI/CD pipeline agents, and application service accounts; integrates with enterprise IdP via SAML 2.0 federation; MFA enforced for all production roles	IBM Cloud IAM — resource group policies, access groups, federated SAML	AWS IAM — roles, instance profiles, OIDC federation for EKS
Load Balancer (Layer 7)	Distributes HTTPS traffic across IKS / EKS node pools using path-based routing; performs health probes every 10 seconds; removes unhealthy nodes within 30 seconds	IBM Cloud Application Load Balancer + IBM CIS global load balancer	AWS Application Load Balancer + Route 53 health-check-driven DNS
DNS & CDN (Global)	Routes users to the nearest healthy cloud region using Anycast DNS; serves static storefront assets from edge PoPs; switches DNS to DR region within 60 seconds of US-South health check failure	IBM Cloud Internet Services (CIS) — Anycast DNS + CDN + WAF	Amazon Route 53 + CloudFront — DR traffic routing target
Hybrid Connectivity	Provides secure, high-throughput private connectivity between on-premises data centers and IBM Cloud for inventory sync and order routing; IPsec VPN serves as automated failover path	IBM Cloud Direct Link 2.0 (10 Gbps primary) + IPsec VPN (failover)	AWS Direct Connect (standby) — activated if IBM Direct Link unavailable

3. Infrastructure Modules

Reusable Terraform modules encapsulate provider-specific configuration details behind a standardized interface of input variables and output values. Each module is maintained in a dedicated directory within the platform's infrastructure repository, versioned with semantic tags, and referenced by environment-level root configurations using the Terraform source attribute. This design means that a change to the network module — for example, adding a new subnet — propagates consistently to every environment that consumes it, without requiring the change to be applied manually in each place.

The five core modules below cover the complete infrastructure surface of the Global Retail Platform across both IBM Cloud and AWS. Each module accepts provider-agnostic inputs wherever possible; provider-specific resource types are isolated within the module body, keeping the calling configuration portable across environments.

Module	Purpose	Key Inputs	Key Outputs
network	Creates a fully segmented virtual network environment per region, including VPC, all subnets,	region, vpc_cidr, subnet_map (name → CIDR), enable_nat_gateway, tags	vpc_id, subnet_ids (map), route_table_ids, nat_gateway_ip

Module	Purpose	Key Inputs	Key Outputs
	route tables, NAT gateways, internet gateways, and network ACLs. Enforces the nine-segment topology (DMZ, API gateway, application, payment, data, hybrid connectivity, management, EU compliance zones) defined in the architecture design.		
compute	Provisions Kubernetes worker node pools (IKS worker pools on IBM Cloud; EKS managed node groups on AWS) with configurable instance types, minimum and maximum replica counts, and auto-scaling policies aligned to the HPA thresholds defined in the resilience strategy.	cluster_name, node_pool_configs (name, instance_type, min, max, labels), kubernetes_version, vpc_id, subnet_ids	cluster_id, cluster_endpoint, kubeconfig, node_pool_ids
database	Deploys managed PostgreSQL and Redis instances with high-availability replication, automated backups, encryption at rest using customer-managed keys from the provider key management service, and private service endpoint access restricted to the data persistence subnet.	engine (postgres redis), instance_class, storage_gb, backup_retention_days, kms_key_id, subnet_ids, allowed_cidr_blocks	endpoint, port, connection_string_secret_id, replica_endpoints
storage	Provisions object storage buckets with configurable access policies, server-side encryption, versioning, lifecycle	bucket_configs (name, versioning, retention_days, worm_enabled), block_volume_configs (name, size_gb, iops_tier), kms_key_id	bucket_names, bucket_arns, volume_ids, volume_attachment_ids

Module	Purpose	Key Inputs	Key Outputs
	transition rules (Standard → Archive after 90 days), and immutable WORM retention locks for audit evidence buckets. Also provisions block storage volumes for database nodes.		
security	Defines all security group rules, Kubernetes NetworkPolicy manifests, IAM roles and policies, service account bindings, and Secrets Manager configuration. Enforces default-deny between all network segments; generates least-privilege IAM role definitions for CI/CD pipeline agents, DevOps operators, and application service accounts.	environment, allowed_ingress_cidrs, payment_processor_egress_ips, iam_role_definitions, mfa_required (bool)	security_group_ids (map), iam_role_arns (map), secrets_manager_arns
observability	Provisions centralized monitoring, log aggregation, and alerting infrastructure. Configures IBM Cloud Monitoring or Amazon CloudWatch dashboards, log ingestion pipelines (Fluent Bit → Log Analysis / CloudWatch Logs), immutable audit log buckets, and PagerDuty alert routing for the anomaly detection rules defined in the compliance strategy.	log_retention_days, alert_endpoints (PagerDuty, Slack), dashboard_configs, critical_services (list)	monitoring_workspace_id, log_analysis_instance_id, alert_policy_ids, audit_bucket_arn

3.1 Module Directory Structure

The infrastructure repository follows a consistent directory layout that separates reusable module definitions from environment-specific root configurations:

```
infrastructure/
modules/
  network/      # VPC, subnets, routing, NAT
  compute/      # IKS / EKS clusters and node pools
  database/     # PostgreSQL, Redis (IBM & AWS)
  storage/      # Object storage, block volumes
  security/     # Security groups, IAM, NetworkPolicy
  observability/ # Monitoring, logging, alerting
environments/
  dev/          # Development: IBM Cloud US-South
  staging/      # Staging: IBM Cloud US-South
  production/  # Production: IBM Cloud US-South (primary)
  eu-compliance/ # GDPR region: IBM Cloud EU-Frankfurt
  dr/          # Disaster recovery: AWS us-east-1
```

4. State and Environment Management

Terraform tracks the state of every provisioned resource in a state file that maps configuration declarations to real infrastructure objects. Correct state management is foundational to safe infrastructure automation: without it, plan operations cannot detect drift, and concurrent executions can corrupt the state file, leading to orphaned or duplicated resources. The Global Retail Platform applies three controls — environment separation, remote state backends, and state locking — across all five deployment environments.

Control	Description	Implementation
Environment Separation	Each environment (dev, staging, production, eu-compliance, dr) maintains its own completely independent Terraform state file. Changes to production infrastructure cannot affect or be confused with development resources, and a failed apply in dev cannot corrupt the production state.	Separate Terraform workspaces and state backend paths per environment: s3://retail-tf-state/dev/, /staging/, /production/, /eu-compliance/, /dr/. Each path is scoped to a dedicated IBM COS bucket (primary) or S3 bucket (AWS DR), with IAM policies restricting write access by environment.
Remote State Backend	State files are stored in secure, versioned, remotely accessible backends rather than on local developer machines or CI/CD agent ephemeral filesystems. This ensures that state is always available for plan and apply operations regardless of which machine executes Terraform, and that historical state versions can	IBM Cloud Object Storage bucket configured as the Terraform remote backend for all IBM Cloud environments (ibm cloud backend provider). AWS S3 with versioning enabled serves as the backend for the AWS DR environment. Both backends enforce AES-256 encryption at rest and restrict access to the CI/CD pipeline service account and DevOps Lead role.

Control	Description	Implementation
	be recovered if a state file is corrupted.	
State Locking	Concurrent Terraform apply operations on the same environment would produce race conditions that corrupt state. State locking prevents this by acquiring an exclusive lock before any operation that modifies state and releasing it upon completion.	IBM Cloud Object Storage does not natively support state locking; a DynamoDB-compatible locking table is implemented using IBM Cloudant for IBM Cloud environments. AWS environments use DynamoDB native state locking. The lock record includes the operation type, operator identity, and timestamp, providing an audit trail of every state modification.
State Versioning & Recovery	State files are treated as critical operational artifacts. Accidental deletion or corruption of a state file would make Terraform unable to manage existing infrastructure without manual import operations.	Object Storage versioning is enabled on all state backend buckets. A maximum of 30 state file versions are retained per environment. Automated integrity checks run nightly; if the current state file fails checksum verification, an alert fires to the DevOps Slack channel within 5 minutes.
Environment Parameter Files	Each environment has distinct values for resource sizes, replica counts, CIDR ranges, and cost tiers. Hard-coding these in module definitions would prevent the same module from serving multiple environments.	Terraform .tfvars files per environment define all variable overrides: dev uses minimal instance types and single-AZ databases; staging mirrors production topology at reduced scale; production and eu-compliance use full multi-zone HA configurations; dr uses AWS resources sized to receive full production traffic on failover activation.
Secrets Management	Infrastructure provisioning requires sensitive credentials (IBM API keys, database passwords, TLS certificates) that must never appear in Terraform state files or version-controlled configuration.	All secrets are read at plan-time from IBM Secrets Manager (IBM Cloud environments) or AWS Secrets Manager (AWS DR environment) using the respective Terraform data sources. Secrets are injected as environment variables for CI/CD pipeline executions and are never written to .tfvars files or committed to Git.

5. Drift Detection Strategy

Configuration drift occurs when the actual state of deployed infrastructure diverges from the desired state defined in Terraform templates. Drift is typically introduced by manual changes made directly through cloud console interfaces, by automated remediation tools that modify resources without updating Terraform state, or by cloud provider maintenance operations. For the Global Retail Platform, undetected drift carries significant risk: a manually added security group rule could silently open a network path into the Payment Isolation Zone, violating PCI-DSS controls without triggering any pipeline alert.

Drift Risk	Detection Mechanism	Mitigation Strategy
Manual infrastructure changes via cloud console	Scheduled terraform plan --detailed-exitcode executed every 6 hours against all production and staging environments; exit code 2 indicates detected differences	IaC-only change policy: all infrastructure modifications must originate from a Terraform apply executed through the CI/CD pipeline. Console access for production environments is restricted to read-only IAM roles. Any drift detected by the scheduled plan is treated as a P1 incident and must be remediated within 4 hours by running a corrective apply through the approved pipeline.
Configuration mismatches between environments	Automated drift comparison using terraform plan output parsed by a custom script that diffs planned changes against a zero-change baseline; alerts fire if planned changes exceed a zero threshold on any security-related resource	Module-driven consistency: because all environments consume the same versioned modules, configuration mismatches between dev and production are structurally limited to intentional variable differences defined in .tfvars files. Any unintended difference is surfaced by the comparison script and blocks the next pipeline run until resolved.
Unauthorized changes by privileged operators	IBM Cloud Activity Tracker and AWS CloudTrail capture every resource modification API call; events are streamed to the centralized IBM Log Analysis instance and trigger a HIGH alert if any infrastructure resource is modified outside of a Terraform pipeline run	Change management enforcement: all infrastructure change requests are submitted as pull requests to the infrastructure Git repository, reviewed by the DevOps Engineering Lead, and merged only after automated plan validation passes. The Terraform pipeline run ID is recorded in the Activity Tracker event, allowing any modification to be traced to its originating PR within seconds.
Provider-initiated resource changes (maintenance, deprecation)	Nightly terraform plan run captures changes introduced by provider maintenance windows, image updates, or deprecated resource versions; output is diff-compared against the previous night's run to surface new drift	Scheduled reconciliation: if the nightly plan detects provider-introduced changes that are compatible with the IaC definition (e.g., a patched base image), the change is accepted and state is refreshed via terraform apply -refresh-only. Changes that conflict with defined configuration (e.g., a resource type deprecation) are escalated to the DevOps Lead for an explicit module update.
Security group or NetworkPolicy rule drift	Dedicated security drift scan using Checkov static analysis tool executed against the Terraform state export and current provider resources; PCI-DSS and GDPR control rules defined as custom Checkov policies	Security baseline enforcement: Checkov policy violations block the next Terraform apply in any environment and generate a HIGH severity alert routed to both the DevOps and Security Slack channels. Payment Isolation Zone and EU Compliance Zone resources are subject to additional nightly Checkov scans independent of the general drift detection cycle.
Terraform state corruption or accidental resource deletion	State file SHA-256 checksum verified before every plan operation; mismatch triggers an	Automated state recovery: if state corruption is confirmed, the previous valid state version is restored from the object storage version history within 15 minutes. Affected resources are then re-

Drift Risk	Detection Mechanism	Mitigation Strategy
	immediate pipeline abort and PagerDuty alert; state version history in object storage enables point-in-time recovery	imported using terraform import to reconcile the recovered state with the actual infrastructure, after which a plan is run to confirm zero drift before resuming normal pipeline operations.

6. Provisioning Validation

Provisioning validation confirms that Terraform has created infrastructure that matches the declared configuration and that the resulting environment is ready to support application workloads. Validation is applied in two phases: post-apply structural checks that verify resource existence and configuration attributes, and functional integration tests that exercise network connectivity, security policy enforcement, and service reachability. Together these phases give the DevOps team confidence that a newly provisioned environment will behave identically to the environment that was defined in code before any application is deployed into it.

Validation Check	Purpose	Implementation
Resource Existence Verification	Confirms that all infrastructure resources defined in the Terraform configuration were successfully created and are present in the provider with the correct identifiers, sizes, and tags	terraform show -json is parsed by a validation script that asserts the presence and attribute values of every expected resource. The script is executed as a mandatory post-apply step in the infrastructure CI/CD pipeline and fails the pipeline if any expected resource is absent or has incorrect attributes such as wrong subnet CIDR, missing encryption flag, or incorrect instance type.
Terraform Plan Zero-Difference Check	Confirms that the infrastructure state after apply exactly matches the desired configuration; a non-zero plan output after apply indicates that an apply step completed with errors or partial application	Immediately following every terraform apply, a terraform plan is executed against the same configuration. If the plan output indicates any planned changes (exit code 2), the pipeline fails and an alert is raised. This check is the primary mechanism for catching partial apply failures where some resources were created and others were not.
Network Connectivity Testing	Ensures that services within each network segment can reach their intended peers and that traffic between non-adjacent segments is correctly blocked by security group and NetworkPolicy rules	Terratest Go test suites execute after provisioning to validate connectivity: (1) application pods in the application services zone can reach PostgreSQL on the expected port; (2) pods in the payment namespace cannot initiate connections to the application zone (default-deny verification); (3) the hybrid connectivity zone can reach on-premises test endpoints via Direct Link; (4) EU Compliance Zone resources

Validation Check	Purpose	Implementation
		cannot establish connections to US-South object storage buckets.
Security Policy Validation	Verifies that IAM roles, security group rules, Kubernetes NetworkPolicies, and Secrets Manager access policies enforce the principle of least privilege as defined in the security module	Checkov policy-as-code scans run against the Terraform plan output before apply (shift-left) and against the deployed state after apply (confirmation). A dedicated suite of custom Checkov rules encodes the platform's compliance requirements: PCI-DSS payment namespace isolation rules, GDPR EU data residency controls, and encryption-at-rest requirements for all storage resources. Any failing rule blocks the pipeline and generates a report sent to the Security and Compliance Director.
Kubernetes Cluster Health Check	Confirms that the provisioned IKS or EKS cluster is reachable, that all node pools are in Ready state, and that the minimum required node count is available before any application deployment is attempted	kubectl get nodes and kubectl get componentstatuses are executed against the new cluster endpoint using the kubeconfig output from the compute module. The validation script asserts that all nodes report Ready status and that the API server, etcd, and scheduler are healthy. If any node is not ready within a configurable timeout (default: 10 minutes), the cluster provisioning is flagged as failed.
Database Connectivity and Encryption Verification	Confirms that provisioned database instances are reachable from the application subnet, that TLS connections are enforced, and that encryption at rest is enabled with the correct customer-managed key	A validation job runs inside the application subnet and attempts a TLS connection to the PostgreSQL and Redis endpoints. The job verifies that: (1) the TLS certificate is issued by the expected CA; (2) plaintext connections are rejected; (3) the IBM Key Protect or AWS KMS key ID on the volume matches the key defined in the Terraform configuration. Key mismatch triggers a CRITICAL alert to the Security and Compliance Director.
Load Balancer and DNS Resolution Check	Confirms that the application load balancer is provisioned with the correct listener rules and that DNS records resolve to the expected load balancer endpoints in each region	The validation script performs DNS resolution against IBM Cloud environments and Route 53 for AWS DR, asserting that each environment-specific hostname resolves to the correct load balancer IP. HTTP health probe responses are then checked against expected status codes (200 OK on the /health endpoint) for each registered service.
Cross-Provider State Consistency Check	Confirms that resources in the IBM Cloud and AWS provider	A cross-provider validation script reads the Terraform output values from both

Validation Check	Purpose	Implementation
	configurations that are interdependent (such as DR replication targets and Direct Connect endpoints) are mutually consistent and reference the correct counterpart resources across providers	the IBM Cloud and AWS environment state backends and asserts referential consistency: the AWS S3 replication target bucket ARN matches the IBM COS replication source configured in the storage module; the AWS VPN gateway endpoint IP matches the peer IP configured in the IBM IPSec VPN resource. Inconsistencies indicate a partial multi-cloud apply and trigger an immediate pipeline failure.

6.1 Validation Pipeline Integration

All six validation checks are integrated as mandatory stages in the infrastructure CI/CD pipeline, which runs separately from but in coordination with the application CI/CD pipeline defined in the product microservice lab. The infrastructure pipeline follows the same IBM Cloud Toolchain execution model and enforces the same promotion gates:

- terraform validate and terraform plan execute on every pull request to the infrastructure repository
- Checkov security scans run as a pre-apply gate; any critical policy failure blocks the apply
- terraform apply executes automatically for dev environment changes; staging and production require manual approval from the DevOps Engineering Lead
- Post-apply validation tests (resource existence, plan zero-diff, connectivity, security, DNS) execute automatically after every apply
- Validation failures trigger an automatic terraform destroy of the failed environment (dev and staging only) and alert the DevOps team; production failures trigger rollback to the previous Terraform state version